Cpp self Learning.

```
1.  #include<iostream>
2.  using namespace std;
3.
4.  // Swap using Call by Value
5.  void swapByValue(int a, int b)
6.  {
7.      int temp;
8.      temp = a;
9.      a = b;
10.     b = temp;
11. }
12.
13. // Swap using Call by Reference
14. void swapByReference(int &a, int &b)
15. {
16.     int temp;
17.     temp = a;
18.     a = b;
19.     b = temp;
20. }
21.
22. int main()
23. {
24.     int num1 = 10;
25.     int num2 = 20;
26.
27.     cout << "Before swapByValue: ";
28.     cout << num1 << " " << num2 << endl;
29.
30.     swapByValue(num1, num2);
31.
32.     cout << "After swapByValue: ";
33.     cout << num1 << " " << num2 << endl;
34.
35.     cout << "\nBefore swapByReference: ";
36.     cout << num1 << " " << num2 << endl;
37.
38.     swapByReference(num1, num2);
```

```
39.
40.     cout << "After swapByReference: ";
41.     cout << num1 << " " << num2 << endl;
42.
43.     return 0;
44. }
```

2. Write an inline function to calculate factorial and calculate power.

```
#include<iostream>
using namespace std;

// Inline function for factorial
inline int factorial(int n) {
   int fact = 1;
   for(int i = 1; i <= n; i++) {
       fact *= i;
   }
   return fact;
}

// Inline function for power
inline int power(int base, int exp) {
   int result = 1;
   for(int i = 1; i <= exp; i++) {
       result *= base;
   }
   return result;
}

int main() {
   int num, base, exp;

   cout << "Enter number for factorial: ";
   cin >> num;
   cout << "Factorial = " << factorial(num) << endl;
```

```
    cout << "\nEnter base and exponent: ";
    cin >> base >> exp;
    cout << "Power = " << power(base, exp) << endl;

    return 0;
}
```

3. Write a program to: Ask the user for number of students. Dynamically allocate an array of Student objects using new. Each student contains: rollNo marks Display: All student records Highest marks Properly release deallocate memory.

```
#include<iostream>
using namespace std;

class Student {
private:
    int rollNo;
    double marks;

public:
    void accept() {
        cout << "Enter Roll No: ";
        cin >> rollNo;
        cout << "Enter Marks: ";
        cin >> marks;
    }

    void display() {
        cout << "Roll No: " << rollNo << ", Marks: " << marks << endl;
    }

    double getMarks() {
        return marks;
    }
};

int main() {
```

```
    int n;
    cout << "Enter number of students: ";
    cin >> n;

    // Dynamic allocation
    Student *arr = new Student[n];

    // Accept data
    cout << "\nEnter student details:\n";
    for(int i = 0; i < n; i++) {
        cout << "\nStudent " << i + 1 << ":\n";
        arr[i].accept();
    }

    // Display all records
    cout << "\n--- Student Records ---\n";
    for(int i = 0; i < n; i++) {
        arr[i].display();
    }

    // Find highest marks
    double maxMarks = arr[0].getMarks();
    for(int i = 1; i < n; i++) {
        if(arr[i].getMarks() > maxMarks) {
            maxMarks = arr[i].getMarks();
        }
    }

    cout << "\nHighest Marks: " << maxMarks << endl;

    // Deallocate memory
    delete[] arr;

    return 0;
}
```

4. Create a class named Laptop with the following specifications: Class Requirements: Private data members: brandId price A parameterless constructor that: Initializes brandId to 1 Initializes price to 50000 Displays the message: "Laptop

Constructor Called" A destructor that displays the message: "Laptop Destructor Called" A member function display() that prints the laptop details.

```
#include<iostream>
using namespace std;

class Laptop {
private:
   int brandId;
   double price;

public:
   // Parameterless constructor
   Laptop() {
      brandId = 1;
      price = 50000;
      cout << "Laptop Constructor Called" << endl;
   }

   // Destructor
   ~Laptop() {
      cout << "Laptop Destructor Called" << endl;
   }

   // Display function
   void display() {
      cout << "Brand ID: " << brandId << endl;
      cout << "Price: " << price << endl;
   }
};

int main() {
   Laptop l1;   // Object created → constructor called

   cout << "\nLaptop Details:\n";
   l1.display();

   return 0;   // Object destroyed → destructor called
}
```

5. Write your own implementations of `strlen()` and `strcpy()` without using `string.h`.

```
#include<iostream>
using namespace std;

// Custom strlen
int myStrlen(const char *str) {
   int count = 0;
   while(str[count] != '\0') {
      count++;
   }
   return count;
}

// Custom strcpy
void myStrcpy(char *dest, const char *src) {
   int i = 0;
   while(src[i] != '\0') {
      dest[i] = src[i];
      i++;
   }
   dest[i] = '\0';  // null terminate
}

int main() {
   char str1[100], str2[100];

   cout << "Enter a string: ";
   cin >> str1;

   // strlen
   int length = myStrlen(str1);
   cout << "Length of string: " << length << endl;

   // strcpy
   myStrcpy(str2, str1);
   cout << "Copied string: " << str2 << endl;
```

```
    return 0;
}
```

6. Create a class BankAccount with private data members (accountNumber, accountHolderName, balance). Provide public member functions to deposit, withdraw, and display balance. Create an object in main() and demonstrate encapsulation by accessing data only through public functions i.e. deposit(), withdraw().

```
#include<iostream>
using namespace std;

class BankAccount {
private:
    int accountNumber;
    string accountHolderName;
    double balance;

public:
    // Constructor
    BankAccount(int accNo, string name, double bal) {
        accountNumber = accNo;
        accountHolderName = name;
        balance = bal;
    }

    // Deposit function
    void deposit(double amount) {
        balance += amount;
        cout << "Amount Deposited: " << amount << endl;
    }

    // Withdraw function
    void withdraw(double amount) {
        if(amount <= balance) {
            balance -= amount;
            cout << "Amount Withdrawn: " << amount << endl;
        } else {
```

```
            cout << "Insufficient Balance!" << endl;
        }
    }

    // Display balance
    void display() {
        cout << "\nAccount Number: " << accountNumber << endl;
        cout << "Account Holder: " << accountHolderName << endl;
        cout << "Balance: " << balance << endl;
    }
};

int main() {
    // Object creation
    BankAccount acc1(101, "Simran", 5000);

    acc1.display();

    // Access through public functions only
    acc1.deposit(2000);
    acc1.withdraw(1500);

    acc1.display();

    return 0;
}
```

7. Create `Student` and `Teacher` class inside a namespace `college`. Write constructors, getter/setters, display() method in both classes. Create objects in main() and call methods.

```
#include<iostream>
using namespace std;

// Namespace
namespace college {

    // -------- Student Class --------
    class Student {
```

```
private:
    int rollNo;
    string name;
    double marks;

public:
    // Constructor
    Student(int r = 0, string n = "NA", double m = 0.0) {
        rollNo = r;
        name = n;
        marks = m;
    }

    // Setters
    void setRollNo(int r) { rollNo = r; }
    void setName(string n) { name = n; }
    void setMarks(double m) { marks = m; }

    // Getters
    int getRollNo() { return rollNo; }
    string getName() { return name; }
    double getMarks() { return marks; }

    // Display
    void display() {
        cout << "Student -> RollNo: " << rollNo
             << ", Name: " << name
             << ", Marks: " << marks << endl;
    }
};

// -------- Teacher Class --------
class Teacher {
private:
    int id;
    string name;
    double salary;

public:
    // Constructor
    Teacher(int i = 0, string n = "NA", double s = 0.0) {
```

```
        id = i;
        name = n;
        salary = s;
    }

    // Setters
    void setId(int i) { id = i; }
    void setName(string n) { name = n; }
    void setSalary(double s) { salary = s; }

    // Getters
    int getId() { return id; }
    string getName() { return name; }
    double getSalary() { return salary; }

    // Display
    void display() {
        cout << "Teacher -> ID: " << id
             << ", Name: " << name
             << ", Salary: " << salary << endl;
    }
  };
}

// -------- Main --------
int main() {
  // Creating objects using namespace
  college::Student s1(1, "Simran", 85.5);
  college::Teacher t1(101, "Mehta", 50000);

  cout << "\n--- Student Details ---\n";
  s1.display();

  cout << "\n--- Teacher Details ---\n";
  t1.display();

  return 0;
}
```

8. Create class Programmer and class Teacher, both containing a function work(). Derive class ProgrammingTeacher from both classes. Create an object of ProgrammingTeacher and call work() to show ambiguity. Resolve the ambiguity using scope resolution operator. Add `cout` statements in constructors to confirm constructor calling sequence.

```
#include<iostream>
using namespace std;

// -------- Programmer --------
class Programmer {
public:
   Programmer() {
      cout << "Programmer Constructor Called\n";
   }

   void work() {
      cout << "Programming work\n";
   }
};

// -------- Teacher --------
class Teacher {
public:
   Teacher() {
      cout << "Teacher Constructor Called\n";
   }

   void work() {
      cout << "Teaching work\n";
   }
};

// -------- Derived Class --------
class ProgrammingTeacher : public Programmer, public Teacher {
public:
   ProgrammingTeacher() {
      cout << "ProgrammingTeacher Constructor Called\n";
```

```
    }
};

int main() {
    ProgrammingTeacher pt;
    cout << "\nResolving Ambiguity:\n";
    pt.Programmer::work();
    pt.Teacher::work();

    return 0;
}
```

9. Create a base class Person with data members name and age and a member function display(). Derive two classes Student (with data member marks) and Teacher (with data member salary) from Person. Add a function in each derived class (study() for Student and teach() for Teacher). Add abstract method work() in Person and implement in both derived classes. Create Student and Teacher object in main() to demonstrate run-time Polymorphism.

```
#include<iostream>
using namespace std;

// -------- Base Class --------
class Person {
protected:
    string name;
    int age;

public:
    Person(string n = "NA", int a = 0) : name(n), age(a) {}

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }

    // Pure virtual function (abstract)
    virtual void work() = 0;
};
```

```
// -------- Student --------
class Student : public Person {
private:
    double marks;

public:
    Student(string n, int a, double m) : Person(n, a), marks(m) {}

    void study() {
        cout << "Student is studying\n";
    }

    void work() {
        cout << "Student studies and prepares for exams\n";
    }

    void display() {
        Person::display();
        cout << "Marks: " << marks << endl;
    }
};

// -------- Teacher --------
class Teacher : public Person {
private:
    double salary;

public:
    Teacher(string n, int a, double s) : Person(n, a), salary(s) {}

    void teach() {
        cout << "Teacher is teaching\n";
    }

    void work() {
        cout << "Teacher teaches students\n";
    }

    void display() {
        Person::display();
```

```
        cout << "Salary: " << salary << endl;
    }
};

// -------- Main --------
int main() {

    // Runtime polymorphism using base class pointer
    Person *p;

    Student s1("Simran", 20, 85.5);
    Teacher t1("Mehta", 40, 50000);

    cout << "\n--- Student ---\n";
    p = &s1;
    p->display();
    p->work();

    cout << "\n--- Teacher ---\n";
    p = &t1;
    p->display();
    p->work();

    return 0;
}
```

10. Implement a `Matrix` class and overload `+`, `-`, and `*` operators.

```
#include<iostream>
using namespace std;

class Matrix {
private:
    int mat[3][3];

public:
    // Accept matrix
    void accept() {
```

```
        cout << "Enter 3x3 matrix:\n";
        for(int i = 0; i < 3; i++) {
            for(int j = 0; j < 3; j++) {
                cin >> mat[i][j];
            }
        }
    }

    // Display matrix
    void display() {
        for(int i = 0; i < 3; i++) {
            for(int j = 0; j < 3; j++) {
                cout << mat[i][j] << " ";
            }
            cout << endl;
        }
    }

    // + operator
    Matrix operator+(Matrix m) {
        Matrix temp;
        for(int i = 0; i < 3; i++) {
            for(int j = 0; j < 3; j++) {
                temp.mat[i][j] = mat[i][j] + m.mat[i][j];
            }
        }
        return temp;
    }

    // - operator
    Matrix operator-(Matrix m) {
        Matrix temp;
        for(int i = 0; i < 3; i++) {
            for(int j = 0; j < 3; j++) {
                temp.mat[i][j] = mat[i][j] - m.mat[i][j];
            }
        }
        return temp;
    }

    // * operator
```

```
    Matrix operator*(Matrix m) {
        Matrix temp;
        for(int i = 0; i < 3; i++) {
            for(int j = 0; j < 3; j++) {
                temp.mat[i][j] = 0;
                for(int k = 0; k < 3; k++) {
                    temp.mat[i][j] += mat[i][k] * m.mat[k][j];
                }
            }
        }
        return temp;
    }
};

// -------- Main --------
int main() {
    Matrix m1, m2, result;

    cout << "Enter Matrix 1:\n";
    m1.accept();

    cout << "Enter Matrix 2:\n";
    m2.accept();

    cout << "\nMatrix 1:\n";
    m1.display();

    cout << "\nMatrix 2:\n";
    m2.display();

    // Addition
    result = m1 + m2;
    cout << "\nAddition:\n";
    result.display();

    // Subtraction
    result = m1 - m2;
    cout << "\nSubtraction:\n";
    result.display();

    // Multiplication
```

```
    result = m1 * m2;
    cout << "\nMultiplication:\n";
    result.display();

    return 0;
}
```

11. Implement a singly linear linked list with operations `add_first()` and `add_last()` to demonstrate proper use of friend class.

```
#include<iostream>
using namespace std;

// -------- Node Class --------
class Node {
private:
    int data;
    Node* next;

public:
    Node(int value) {
        data = value;
        next = NULL;
    }

    // Declare LinkedList as friend
    friend class LinkedList;
};

// -------- LinkedList Class --------
class LinkedList {
private:
    Node* head;

public:
    LinkedList() {
```

```
        head = NULL;
    }

    // Add at beginning
    void add_first(int value) {
        Node* newNode = new Node(value);

        newNode->next = head;
        head = newNode;
    }

    // Add at end
    void add_last(int value) {
        Node* newNode = new Node(value);

        if(head == NULL) {
            head = newNode;
            return;
        }

        Node* temp = head;
        while(temp->next != NULL) {
            temp = temp->next;
        }

        temp->next = newNode;
    }

    // Display list
    void display() {
        Node* temp = head;
        while(temp != NULL) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL\n";
    }
};

// -------- Main --------
int main() {
```

```
    LinkedList list;

    list.add_first(10);
    list.add_first(20);
    list.add_last(30);
    list.add_last(40);

    cout << "Linked List:\n";
    list.display();

    return 0;
}
```

12. Implement a `Product` (fields: title, price & virtual methods accept() & display()) class with derived classes `Book` (fields: pages) and `Tape` (fields: playtime). Create an array of 5 Product pointers and initialize it with Product & Tape objects as follows. Write a loop to display only pages and playtime values. Product *arr = new Product*[5]; for(int i=0; i<< "1. Book, 2. Tape, Enter choice: "; cin >> choice; switch(choice) { case 1: arr[i] = new Book; arr[i]->accept(); break; case 2: arr[i] = new Tape; arr[i]->accept(); break; } }

```
#include<iostream>
using namespace std;

// -------- Base Class --------
class Product {
protected:
    string title;
    double price;

public:
    virtual void accept() {
        cout << "Enter Title: ";
        cin >> title;
        cout << "Enter Price: ";
        cin >> price;
    }

    virtual void display() {
```

```
        cout << "Title: " << title << ", Price: " << price << endl;
    }

    virtual ~Product() {}
};

// -------- Book --------
class Book : public Product {
private:
    int pages;

public:
    void accept() {
        Product::accept();
        cout << "Enter Pages: ";
        cin >> pages;
    }

    int getPages() {
        return pages;
    }
};

// -------- Tape --------
class Tape : public Product {
private:
    int playtime;

public:
    void accept() {
        Product::accept();
        cout << "Enter Playtime: ";
        cin >> playtime;
    }

    int getPlaytime() {
        return playtime;
    }
};

// -------- Main --------
```

```
int main() {
    int choice;

    // As given in question
    Product *arr[5];

    // Initialization
    for(int i = 0; i < 5; i++) {
        cout << "\n1. Book, 2. Tape, Enter choice: ";
        cin >> choice;

        switch(choice) {
            case 1:
                arr[i] = new Book;
                arr[i]->accept();
                break;

            case 2:
                arr[i] = new Tape;
                arr[i]->accept();
                break;

            default:
                cout << "Invalid choice\n";
                i--; // retry same index
        }
    }

    // Display only pages & playtime
    cout << "\n--- Required Output ---\n";

    for(int i = 0; i < 5; i++) {
        if(Book* b = dynamic_cast<Book*>(arr[i])) {
            cout << "Pages: " << b->getPages() << endl;
        }
        else if(Tape* t = dynamic_cast<Tape*>(arr[i])) {
            cout << "Playtime: " << t->getPlaytime() << endl;
        }
    }

    // Free memory
```

```
    for(int i = 0; i < 5; i++) {
        delete arr[i];
    }

    return 0;
}
```

13. Create a `Payment` base class and derived classes `CreditCard`, `UPI`, and `Cash`. Write method `void makePayment(double amount)` in base class (choose virtual or abstract) and override in derived classes (Simulate payment with appropriate `cout` statements). Write a 3 of 4 static method `Payment* create(char *mode)` in a new `PaymentFactory` class. In main() get payment object via factory and test its makePayment().

```
#include<iostream>
#include<cstring>
using namespace std;

// -------- Base Class --------
class Payment {
public:
    virtual void makePayment(double amount) = 0; // pure virtual

    virtual ~Payment() {}
};

// -------- CreditCard --------
class CreditCard : public Payment {
public:
    void makePayment(double amount) {
        cout << "Payment of " << amount
             << " done using Credit Card\n";
    }
};

// -------- UPI --------
class UPI : public Payment {
public:
```

```
    void makePayment(double amount) {
        cout << "Payment of " << amount
             << " done using UPI\n";
    }
};

// -------- Cash --------
class Cash : public Payment {
public:
    void makePayment(double amount) {
        cout << "Payment of " << amount
             << " done using Cash\n";
    }
};

// -------- Factory Class --------
class PaymentFactory {
public:
    static Payment* create(char *mode) {
        if(strcmp(mode, "card") == 0)
            return new CreditCard();
        else if(strcmp(mode, "upi") == 0)
            return new UPI();
        else if(strcmp(mode, "cash") == 0)
            return new Cash();
        else
            return NULL;
    }
};

// -------- Main --------
int main() {
    char mode[10];
    double amount;

    cout << "Enter payment mode (card/upi/cash): ";
    cin >> mode;

    cout << "Enter amount: ";
    cin >> amount;
```

```
    // Get object using factory
    Payment *p = PaymentFactory::create(mode);

    if(p != NULL) {
        p->makePayment(amount);
        delete p;   // free memory
    } else {
        cout << "Invalid payment mode\n";
    }

    return 0;
}
```

14. Extend the BankAccount class (Assign 6) to include exception handling: in the deposit() and withdraw() functions, if the amount is negative, throw the amount as a double, and in the withdraw() function, if the amount is greater than the current balance, throw an error message as a string. In main(), use multiple catch blocks to handle these exceptions and display appropriate messages when they are caught.

```
#include<iostream>
using namespace std;

class BankAccount {
private:
    int accountNumber;
    string accountHolderName;
    double balance;

public:
    BankAccount(int accNo, string name, double bal) {
        accountNumber = accNo;
        accountHolderName = name;
        balance = bal;
    }

    // Deposit
    void deposit(double amount) {
        if(amount < 0)
```

```
        throw amount;   // throw double

    balance += amount;
    cout << "Deposited: " << amount << endl;
  }

  // Withdraw
  void withdraw(double amount) {
    if(amount < 0)
        throw amount;   // throw double

    if(amount > balance)
        throw string("Insufficient Balance");  // throw string

    balance -= amount;
    cout << "Withdrawn: " << amount << endl;
  }

  void display() {
    cout << "\nAccount Number: " << accountNumber << endl;
    cout << "Name: " << accountHolderName << endl;
    cout << "Balance: " << balance << endl;
  }
};

// -------- Main --------
int main() {
  BankAccount acc(101, "Simran", 5000);

  double amount;

  try {
    cout << "Enter amount to deposit: ";
    cin >> amount;
    acc.deposit(amount);

    cout << "Enter amount to withdraw: ";
    cin >> amount;
    acc.withdraw(amount);
  }
  catch(double amt) {
```

```
        cout << "Exception: Negative amount entered -> " << amt << endl;
    }
    catch(string msg) {
        cout << "Exception: " << msg << endl;
    }

    acc.display();

    return 0;
}
```

15. Throw a custom exception class from `Student` (Assign 7) class methods to prevent negative marks.

```
#include<iostream>
using namespace std;

// -------- Custom Exception Class --------
class InvalidMarksException {
private:
    string message;

public:
    InvalidMarksException(string msg) {
        message = msg;
    }

    string getMessage() {
        return message;
    }
};

// -------- Student Class --------
class Student {
private:
    int rollNo;
    string name;
    double marks;
```

```
public:
    Student(int r = 0, string n = "NA", double m = 0.0) {
        rollNo = r;
        name = n;
        setMarks(m);   // validation here
    }

    void setMarks(double m) {
        if(m < 0)
            throw InvalidMarksException("Marks cannot be negative!");
        marks = m;
    }

    void display() {
        cout << "Roll No: " << rollNo << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
};

// -------- Main --------
int main() {
    try {
        Student s1(1, "Simran", 85);   // valid
        s1.display();

        cout << "\nCreating student with negative marks...\n";

        Student s2(2, "Rahul", -10);  // will throw exception
        s2.display();
    }
    catch(InvalidMarksException e) {
        cout << "Exception: " << e.getMessage() << endl;
    }

    return 0;
}
```

16. What problem you see in the following code in exception stack unwinding? Resolve them using user-defined smart pointer. try { Student *s = new Student; // accept info from user and set using setters s->display(); delete s; } catch(...) { cout << "invalid input" << endl; }

```
#include<iostream>
using namespace std;

class Student {
public:
   Student() {
       cout << "Student Created\n";
   }

   void display() {
       cout << "Student Display\n";
   }

   ~Student() {
       cout << "Student Destroyed\n";
   }
};

// -------- Smart Pointer --------
class SmartPtr {
private:
   Student *ptr;

public:
   SmartPtr(Student *p = NULL) {
       ptr = p;
   }

   // Overload -> operator
   Student* operator->() {
       return ptr;
   }

   // Destructor (automatic cleanup)
   ~SmartPtr() {
```

```
        delete ptr;
        cout << "Memory Freed by SmartPtr\n";
    }
};

// -------- Main --------
int main() {
    try {
        SmartPtr s(new Student());  // smart pointer used

        s->display();

        // Suppose exception occurs here
        throw 1;

    }
    catch(...) {
        cout << "Invalid input\n";
    }

    return 0;
}
```

17. Write a program to save and load `Student` object using file IO.

```
#include<iostream>
#include<fstream>
using namespace std;

class Student {
private:
    int rollNo;
    char name[50];
    double marks;

public:
    void accept() {
        cout << "Enter Roll No: ";
        cin >> rollNo;
```

```
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Marks: ";
        cin >> marks;
    }

    void display() {
        cout << "\nRoll No: " << rollNo << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
};

int main() {
    Student s1;

    // -------- Save to file --------
    ofstream fout("student.dat", ios::binary);

    cout << "--- Enter Student Data ---\n";
    s1.accept();

    fout.write((char*)&s1, sizeof(s1));
    fout.close();

    // -------- Load from file --------
    Student s2;
    ifstream fin("student.dat", ios::binary);

    fin.read((char*)&s2, sizeof(s2));
    fin.close();

    cout << "\n--- Data Read From File ---\n";
    s2.display();

    return 0;
}
```

18. Write a generic class for `Array` to store user-defined data types using templates. Overload required operators. Prevent memory leakage.

```
#include<iostream>
using namespace std;

template <class T>
class Array {
private:
   T *arr;
   int size;

public:
   // Constructor
   Array(int s = 0) {
      size = s;
      arr = new T[size];
   }

   // Copy Constructor (important)
   Array(const Array &other) {
      size = other.size;
      arr = new T[size];
      for(int i = 0; i < size; i++) {
         arr[i] = other.arr[i];
      }
   }

   // Assignment Operator
   Array& operator=(const Array &other) {
      if(this != &other) {
         delete[] arr;
         size = other.size;
         arr = new T[size];
         for(int i = 0; i < size; i++) {
            arr[i] = other.arr[i];
         }
      }
      return *this;
   }

   // Destructor (prevents memory leak)
   ~Array() {
```

```
        delete[] arr;
    }

    // Overload [] operator
    T& operator[](int index) {
        if(index < 0 || index >= size) {
            cout << "Index out of bounds\n";
            exit(0);
        }
        return arr[index];
    }

    // Accept values
    void accept() {
        for(int i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

    // Display values
    void display() {
        for(int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    // Friend << operator
    friend ostream& operator<<(ostream &out, Array<T> &a) {
        for(int i = 0; i < a.size; i++) {
            out << a.arr[i] << " ";
        }
        return out;
    }
};

// -------- Main --------
int main() {
    int n;
    cout << "Enter size: ";
    cin >> n;
```

```
    Array<int> a(n);   // can also use user-defined types

    cout << "Enter elements:\n";
    a.accept();

    cout << "\nArray using display(): ";
    a.display();

    cout << "Array using << : " << a << endl;

    return 0;
}
```

19. Create `list` and store 10 random numbers. Do the following operations one after another. - Display list in reverse order using iterator. - Increment each number in the list by 5 using iterator. - Display list in using const iterator. - Sort the list using appropriate STL algorithm. - Display modified list using default iterator.

```
#include<iostream>
#include<list>
#include<algorithm>
#include<cstdlib>   // for rand()
using namespace std;

int main() {
    list<int> lst;

    // Insert 10 random numbers
    cout << "Random Numbers:\n";
    for(int i = 0; i < 10; i++) {
        int num = rand() % 100;   // 0–99
        lst.push_back(num);
        cout << num << " ";
    }

    // 1. Display in reverse using iterator
    cout << "\n\nReverse Order:\n";
```

```
    for(list<int>::reverse_iterator it = lst.rbegin(); it != lst.rend(); ++it) {
        cout << *it << " ";
    }

    // 2. Increment each element by 5
    for(list<int>::iterator it = lst.begin(); it != lst.end(); ++it) {
        *it = *it + 5;
    }

    // 3. Display using const iterator
    cout << "\n\nAfter Increment (+5):\n";
    for(list<int>::const_iterator it = lst.begin(); it != lst.end(); ++it) {
        cout << *it << " ";
    }

    // 4. Sort the list
    lst.sort();   // list has its own sort()

    // 5. Display using default iterator
    cout << "\n\nSorted List:\n";
    for(auto it = lst.begin(); it != lst.end(); ++it) {
        cout << *it << " ";
    }

    cout << endl;
    return 0;
}
```

20. Demonstrate ownership transfer with unique_ptr, shared ownership with shared_ptr, and non-owning reference with weak_ptr.

```
#include<iostream>
#include<memory>
using namespace std;

class Demo {
public:
    Demo() { cout << "Object Created\n"; }
```

```
    ~Demo() { cout << "Object Destroyed\n"; }

    void show() {
        cout << "Demo Function\n";
    }
};

int main() {

    // -------- unique_ptr (ownership transfer) --------
    cout << "\n--- unique_ptr Demo ---\n";

    unique_ptr<Demo> p1 = make_unique<Demo>();

    // Transfer ownership
    unique_ptr<Demo> p2 = move(p1);

    if(!p1)
        cout << "p1 is now NULL after transfer\n";

    p2->show();


    // -------- shared_ptr (shared ownership) --------
    cout << "\n--- shared_ptr Demo ---\n";

    shared_ptr<Demo> s1 = make_shared<Demo>();
    shared_ptr<Demo> s2 = s1;   // shared ownership

    cout << "Use count: " << s1.use_count() << endl;

    s2->show();


    // -------- weak_ptr (non-owning reference) --------
    cout << "\n--- weak_ptr Demo ---\n";

    weak_ptr<Demo> w1 = s1;   // does not increase count

    cout << "Use count after weak_ptr: " << s1.use_count() << endl;
```

```
    // Access using lock()
    if(shared_ptr<Demo> temp = w1.lock()) {
        temp->show();
    } else {
        cout << "Object no longer exists\n";
    }

    return 0;
}
```